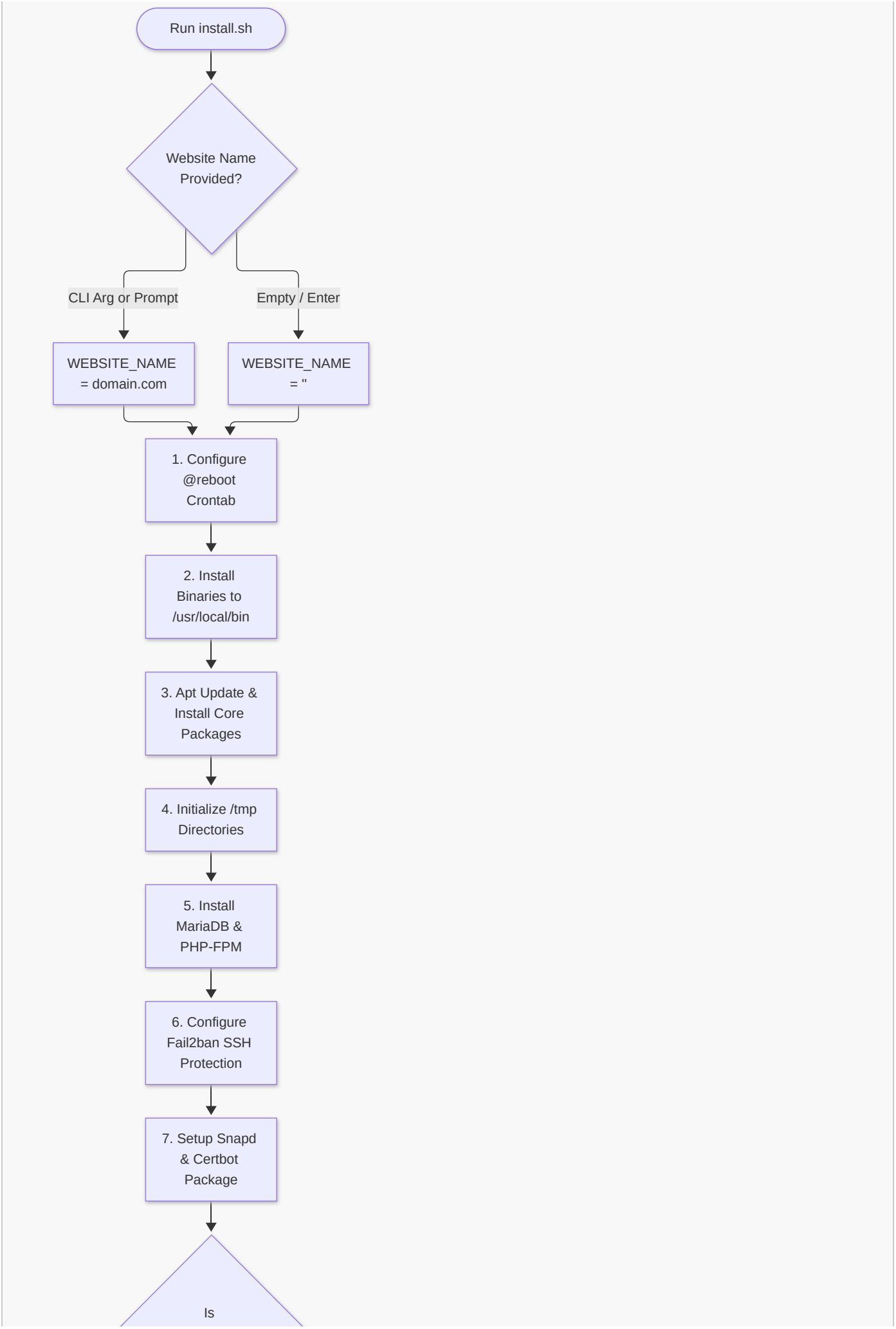
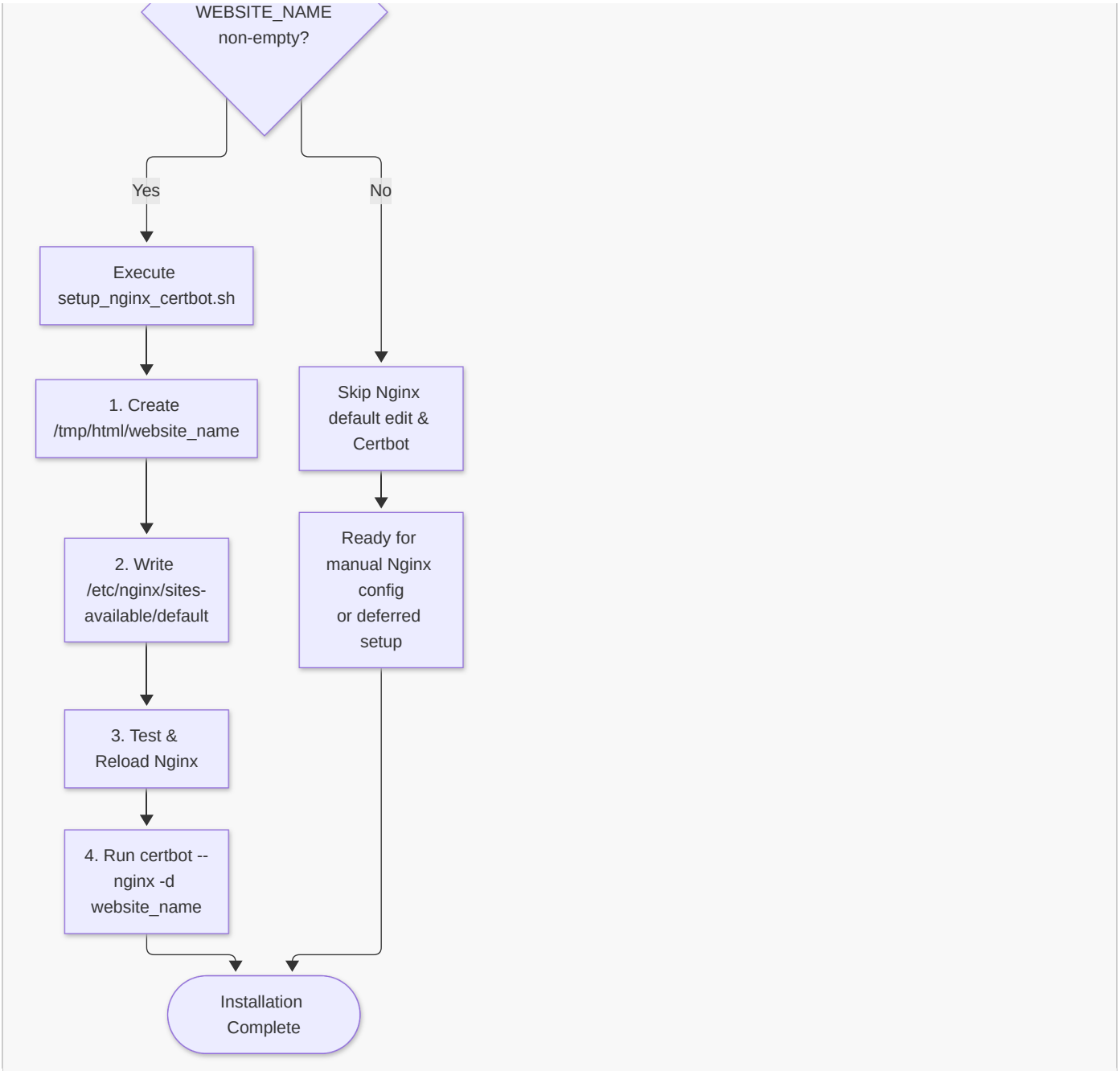
Remote Server Installation Process & Website Variable Guide

This document outlines the architecture, step-by-step workflow, and variable logic for the server provisioning scripts in this repository.

1. High-Level Architecture & Workflow

The installation suite automates system packages, directory structures, security rules, PHP/database runtimes, and web hosting configurations on a Debian/Ubuntu remote server.


Run install.sh
Website Name Provided?
CLI Arg or Prompt
Empty / Enter
WEBSITE_NAME = domain.com
WEBSITE_NAME = ''
1. Configure @reboot Crontab
2. Install Binaries to /usr/local/bin
3. Apt Update & Install Core Packages
4. Initialize /tmp Directories
5. Install MariaDB & PHP-FPM
6. Configure Fail2ban SSH Protection
7. Setup Snapd & Certbot Package
Is



---

2. The `WEBSITE_NAME` Variable

The `WEBSITE_NAME` variable controls whether automated web serving (Nginx) and SSL provisioning (Certbot) are performed during the installation, or deferred for manual/custom setup.

How `WEBSITE_NAME` is Supplied

The variable can be specified in two ways:

1. **Command-Line Argument**:

```
./install.sh example.com
```

2. **Interactive Prompt**: If no argument is passed, [install.sh](file:///var/www/html/remote_server_installation/install.sh) displays an explanation and prompts:

```
Enter website name (or press Enter to skip):
```

Note: Input is automatically sanitized to remove protocol prefixes (`http://`, `https://`), trailing slashes, and whitespace.

---

3. Case-by-Case Breakdown: How `WEBSITE_NAME` Affects Procedures

Procedure / Script	When `WEBSITE_NAME` is Specified (`example.com`)	When `WEBSITE_N` is NOT Specified (
Directory Initialization ([install_temporary_directories.sh](file:///var/www/html/remote_server_installation/install_temporary_directories.sh))	Creates `/tmp/html`, `/tmp/log/nginx`, and the site directory `/tmp/html/example.com` with `755` permissions. Unpacks `html.tar` if present.	Creates base `/tmp/html` and `/tmp/log/nginx` Unpacks `html.ta` present. Does not create a named sit folder.
Nginx Default Configuration ([setup_nginx_certbot.sh](file:///var/www/html/remote_server_installation/setup_nginx_certbot.sh))	**Executed**: Backs up original config to `default_0`. Generates `/etc/nginx/sites-available/default` (and local [default](file:///var/www/html/remote_server_installation/default)) configured to serve from `/tmp/html/example.com`, tests syntax (`nginx -t`), and reloads Nginx.	**Skipped**: `/etc/nginx/sit available/defa` is left untouched. ` can configure virtu hosts or default settings manually.
Certbot SSL Registration ([setup_nginx_certbot.sh](file:///var/www/html/remote_server_installation/setup_nginx_certbot.sh))	**Executed**: Runs `certbot --nginx -d example.com` to request and bind an SSL certificate automatically.	**Skipped**: Certbot i not executed, preventing empty- domain failures or unwanted interact prompts.
System Packages & Security ([install.sh](file:///var/www/html/remote_server_installation/install.sh))	**Executed**: Installs Nginx, Snapd, Certbot binary, Fail2ban with SSH jail rules, and helper scripts in `/usr/local/bin`.	**Executed**: Identica base setup is performed. The se environment is full installed and secur
Database & PHP ([install_mariadb_php_fpm.sh](file:///var/www/html/remote_server_installation/install_mariadb_php_fpm.sh))	**Executed**: Installs MariaDB server, PHP-FPM, PHP-MySQL, configures `date.timezone = "America/New_York"`, and starts services.	**Executed**: Identica Database and PHP runtime are ready regardless of site name.

4. Deep Dive into Connected Procedures

The procedures for **updating Nginx** and **running Certbot** are tightly coupled because:

1. Certbot requires a valid Nginx `server_name` and document `root` to complete ACME HTTP-01 or Nginx-plugin challenges.
2. The Nginx document root must match the directory `/tmp/html/${WEBSITE_NAME}` where site assets reside.

Both steps are encapsulated in [setup_nginx_certbot.sh](file:///var/www/html/remote_server_installation/setup_nginx_certbot.sh).

Procedure Details in [setup_nginx_certbot.sh](file:///var/www/html/remote_server_installation/setup_nginx_certbot.sh):

1. **Document Root Creation**: Creates `/tmp/html/${WEBSITE_NAME}` and applies `chmod 755` so the `www-data` web server user can read files. If the directory is empty, a lightweight placeholder `index.html` is created.
2. **PHP-FPM Socket Detection**: Automatically detects the active PHP version (e.g. `8.2` or `8.3`) to write the correct `fastcgi_pass unix:/run/php/php<version>-fpm.sock;` directive.
3. **Nginx Configuration Generation**: Generates `/etc/nginx/sites-available/default` with:
   - `server_name ${WEBSITE_NAME};`
   - `access_log /tmp/log/nginx/${WEBSITE_NAME}.log;`
   - `root /tmp/html/${WEBSITE_NAME};`
   - FastCGI PHP configuration matching the installed PHP version. Enables the site via symlink in `/etc/nginx/sites-enabled/` and tests with `nginx -t`.
4. **Certbot Execution**: Calls `certbot --nginx -d "${WEBSITE_NAME}"`.

5. Running the Connected Setup at a Later Time

If a user runs [install.sh](file:///var/www/html/remote_server_installation/install.sh) without specifying a website name, they can run the connected Nginx and Certbot configuration whenever they are ready:

```
sudo ./setup_nginx_certbot.sh mynewsite.com
```

Or run interactively:

```
sudo ./setup_nginx_certbot.sh
```

---

6. Database Provisioning Note

`mariadb-server` is installed in place of `mysql-server`. However, `mysql_secure_installation` and database table creation are excluded from these scripts because database creation and schema migrations are handled independently by a separate custom script.